Анимации

Анимации — как специи.
Немного — усиливают вкус. Слишком много — портят всё.

Сергей Дунаев | Архитектор

Сергей Дунаев

Архитектор, Социальная платформа

О себе

- Закончил МЭИ, Автоматика и вычислительная техника
- ВШЭ, Фондовый рынок и инвестиции
  В iOS с 2012 года
- Основатель онлайн-школы SwiftLab
- Хоби: дайвинг, горные лыжи, фортепиано

Ключевые моменты

- Введение
- Базовая анимация UIView
- Анимация трансформаций
- Знакомство с Core Animation
- Отрисовка тени

Что такое анимация в UI?

Это плавное изменение состояния интерфейса во времени.

Примеры:

- Кнопка увеличивается при нажатии
- Картинка “выплывает” на экран
- Экран меняется с эффектом сдвига

Почему анимации важны?

- **Улучшают восприятие интерфейса** — делают его живым и отзывчивым
- **Помогают ориентироваться** — показывают, что произошло (например, открытие экрана)
- **Привлекают внимание** — к важным элементам, новым данным, кнопкам
- **Добавляют удовольствия от использования** — особенно в приложениях с визуальным контенто

Анимации UIView и трансформаций

UIView.animate(withDuration:)

Самый простой способ анимировать свойства UIView. Работает только с **анимируемыми свойствами** (например: frame, alpha, transform, backgroundColor, center и т. д.)

Примеры:

- Анимация длится 2 секунды
- center.x меняется плавно

```
UIView.animate(withDuration: 2) {
   newView.center.x += 200
}
```

- Анимация длится 2 секунды
- Изменение прозрачности и цвета

```
UIView.animate(withDuration: 2) {
   newView.alpha = 0.5
   newView.backgroundColor = .systemRed
}
```

Важные замечания:

- Все анимации работают только на **главном потоке**
- Анимации не остановят текущие действия, а **происходят параллельно**
- UIView.animate **НЕ может анимировать кастомные свойства** (только встроенные)

Параметры анимации и опции AnimationOptions

Основные параметры анимации

```
UIView.animate(
   withDuration: 1.0,
   delay: 0.3,
   options: [.curveEaseInOut],
   animations: {
      // изменения свойств
   },
   completion: { finished in
      // завершение анимации
   }
)
```

Параметр	Описание
`withDuration`	Выполнение анимации в секундах
`delay`	Задержка перед началом анимации
`options`	Поведение анимации (скорость, зацикливание и т.д.)
`animations`	Блок кода, в котором изменяются свойства
`completion`	Замыкание, вызываемое после анимации

Примеры опций UIView.AnimationOptions

Описание	Эффект
`.curveEaseInOut`	Плавное ускорение и замедление (по умолчанию)
`.curveEaseIn`	Быстрое начало → медленный конец
`.curveEaseOut`	Медленное начало → быстрое окончание
`.curveLinear`	Равномерная скорость
`.repeat`	Бесконечный повтор
`.autoreverse`	Анимация проигрывается в обратную сторону
`.allowUserInteraction`	Не блокирует взаимодействие с вью во время анимации
`.beginFromCurrentState`	Продолжает анимацию с текущего состояния (если предыдущая не завершена)

Анимация AutoLayout

При использовании Auto Layout (constraints), мы не можем напрямую изменять frame или center, потому что они управляются системой.
Чтобы корректно анимировать такие элементы, нужно:

- изменять **значения констрейнтов**
- затем вызывать **`layoutIfNeeded()`** внутри блока **`UIView.animate`**

⚠️ **Ошибки, которые делают новички:**

- Меняют constraint, но не вызывают layoutIfNeeded()
  - результат: ничего не анимируется, просто “прыгает”
- Вызывают layoutIfNeeded() вне блока animate
  - результат: все меняется мгновенно

Изменение отступа у вью:

- Меняется значение constant у ограничения
- **`layoutIfNeeded()`** говорит Auto Layout “пересчитай все прямо сейчас”
- Анимация происходит плавно

```
myTopConstraint.constant = 80

UIView.animate(withDuration: 0.4) {
   self.view.layoutIfNeeded()
}
```

Пружинные анимации

Эффект “отскока” и реалистичной физики

Это специальный тип UIView.animate, который использует параметры, имитирующие поведение реальной пружины

Подходит для:

- анимации появления (bounce)
- открытия карточек
- нажатий на кнопки
- оживления интерфейса

Сигнатура метода:

```
UIView.animate(
    withDuration: TimeInterval,
    delay: TimeInterval,
    usingSpringWithDamping: CGFloat,
    initialSpringVelocity: CGFloat,
    options: UIView.AnimationOptions = [],
    animations: () -> Void,
    completion: ((Bool) -> Void)? = nil
)
```

- **`usingSpringWithDamping`** – Коэффициент затухания (0 – сильно пружинит, 1 – без отскока)
- **`initialSpringVelocity`** – Начльная скорость (0 – с нуля, >1 – быстро в начале)

🔍 Подбор параметров:

- Сильный отскок: **`damping: 0.4, velocity: 1.0`**
- Плавное возвращение: **`damping: 0.8, velocity: 0.3`**
- “Мягкое появление”: **`damping: 1.0, velocity: 0`**

Анимация трансформаций

Что такое transform?

Изменяет форму, размер и положение

- Свойство `view.transform` управляет **геометрическим преобразованием вью**
- Использует структуру `CGAffineTransform`
- **Не изменяет** реальные `frame/bounds`, только визуальное представление

⚠️ **Важно:**

- `transform` работает поверх Auto Layout
- Иногда проще использовать `center/frame`, если нужны точные размеры
- При множественных преобразованиях используйте `.concatenating(...)` вместо “перезаписывания”

Комбинирование трансформаций:

```
let move = CGAffineTransform(translationX: 50, y: 0)
let rotate = CGAffineTransform(rotationAngle: .pi / 4)
let scale = CGAffineTransform(scaleX: 1.2, y: 1.2)

let combined = move
    .concatenating(rotate).concatenating(scale)
```

Увеличиваем и поворачиваем вью:

```
UIView.animate(withDuration: 0.4) {
    self.myView.transform = CGAffineTransform(
        scaleX: 1.5,
        y: 1.5
    ).rotated(by: .pi / 6)
}
```

Возврат к исходному состоянию:

```
UIView.animate(withDuration: 0.3) {
    self.myView.transform = .identity
}
```

Знакомство с Core Animation и CABasicAnimation

Core Animation и CABasicAnimation

Больше контроля, плавности и эффектов с Core Animation

Что такое Core Animation?

- Это низкоуровневая система анимаций в iOS
- Работает с **CALayer**, а не с UIView
- Позволяет анимировать практически любое свойство слоя (opacity, position, transform, strokeEnd, и т. д.)
- Можно вызывать анимации **без блоков UIView.animate**

Зачем CABasicAnimation?

- Хочешь больше контроля над:
  - кривыми движения
  - таймингом
  - анимацией CALayer
- Нужно анимировать свойства, которые UIView.animate не покрывает (например, strokeEnd, borderWidth, cornerRadius, shadowOpacity)
- Работаешь с кастомными UI, графикой, CAShapeLayer, CAGradientLayer и т. д.

Что такое CALayer?

Это низкоуровневый графический слой,
который стоит под капотом каждого UIView

Каждый **`UIView`** содержит **`layer`**, который отвечает за:

- отрисовку
- тень
- скругление углов
- прозрачность
- трансформации
- анимации

UIView	CALayer
Участвует в иерархии View	Часть графической иерархии
Обрабатывает касания	Только отрисовка
Используется для UI-логики	Используется для графики
Содержит layer	Является базовым элементом

Полезные свойства:

Свойство	Что делает
`cornerRadius`	Скругление углов
`shadowOpacity`	Прозрачность тени
`shadowColor`	Цвет тени
`borderWidth`	Ширина обводки
`transform`	Масштаб, поворот, сдвиг
`backgroundColor`	Цвет фона
`mask`	Маска отображения (например, по форме)
`contents`	Содержимое (например, CGImage)

Типы слоев на базе CALayer

Класс	Назначение и применение
CALayer	Базовый слой, используется по умолчанию во всех UIView. Отрисовка фона, границ, тени, углы
CAShapeLayer	Для отрисовки фигур и линий (UIBezierPath) **Пример:** круговой прогресс, кастомные иконки, анимированные линии
CAGradientLayer	Для создания градиентов (линейных, радиальных и т. д.) **Пример:** фоны, градиентные прогресс-бары, лоадеры
CATextLayer	Рендер текста прямо в слое (эффективнее, чем UILabel в простых случаях) **Пример:** отрисовка текста в слое без использования UILabel
CAReplicatorLayer	Дублирует подслои с трансформацией (для эффектов, повторений) **Пример:** создание эффектов волны, загрузки, клонов
CAEmitterLayer	Эмиттер частиц (дождь, конфетти, снег и т. д.) **Пример:** конфетти по нажатию кнопки или анимация пыли/снега
CATiledLayer	Используется для отображения больших изображений по частям (тайлами)

Когда использовать эти слои:

✅ Нужна максимальная производительность
✅ Интерфейс содержит динамическую графику
✅ Нужно высококачественное сглаживание и прозрачность
✅ Работаешь с кастомными компонентами UI без логики касаний

CAShapeLayer

- Подкласс CALayer, который умеет отрисовывать произвольные векторные фигуры
- Используется с UIBezierPath или CGPath
- Идеально подходит для: прогресс-баров, рамок, кнопок, индикаторов, линий, кругов и т. д.

Свойство	Описание
`path`	Форма фигуры (CGPath)
`strokeColor`	Цвет линии по контуру
`fillColor`	Цвет заливки
`lineWidth`	Толщина линии
`lineCap`	Концы линии (butt, round, square)
`strokeBegin`	Начало отрисовки (0…1)
`strokeEnd`	Конец отрисовки (0…1)

Пример: круговой прогресс

```
let shapeLayer = CAShapeLayer()
shapeLayer.path = UIBezierPath(
    arcCenter: center,
    radius: 50,
    startAngle: -.pi / 2,
    endAngle: .pi * 1.5,
    clockwise: true
).cgPath
shapeLayer.strokeColor = UIColor.blue.cgColor
shapeLayer.lineWidth = 10
shapeLayer.fillColor = UIColor.clear.cgColor
shapeLayer.strokeEnd = 0.75

view.layer.addSublayer(shapeLayer)
```

Применение:

- Отлично сочетается с CABasicAnimation(keyPath: "strokeEnd") — можно сделать прогресс, таймер, индикатор
- Может использоваться как маска (например, в CAGradientLayer.mask)
- Работает быстрее, чем отрисовка вручную в draw(_:)

CAGradientLayer

- Подкласс CALayer, который рисует градиент
- Поддерживает линейные, радиальные и конусные градиенты

Свойство	Описание
`colors`	Массив цветов градиента (CGColor)
`locations`	Где начинается каждый цвет (0…1)
`startPoint`	Точка начала градиента (в относительных координатах: 0…1)
`endPoint`	Точка конца градиента
`type`	Тип: .axial, .radial, .conic

Пример: горизонтальный градиент

```
let gradient = CAGradientLayer()
gradient.frame = view.bounds
gradient.colors = [
   UIColor.red.cgColor,
   UIColor.orange.cgColor
]
gradient.startPoint = CGPoint(x: 0, y: 0.5)
gradient.endPoint = CGPoint(x: 1, y: 0.5)

view.layer.addSublayer(gradient)
```

Применение:

- Можно использовать анимации colors для смены градиента
- Можно добавить маску (mask = shapeLayer) — и получить градиентный прогресс
- Лучше работает, чем градиент через UIView.draw(_:)в draw(_:)

CABasicAnimation

Анимация strokeEnd (как у прогресс-бара)

```
let animation = CABasicAnimation(keyPath: "strokeEnd")
animation.fromValue = 0
animation.toValue = 1
animation.duration = 1.0

shapeLayer.add(animation, forKey: "strokeAnimation")
```

Параметры:

Параметр	Описание
`keyPath`	Что именно анимируется (“opacity”, “strokeEnd” и т.д.)
`fromValue`	Начальное значение анимации
`toValue`	Конечно значение
`duration`	Длительность
`timeFunction`	Темп движения (по умолчанию линейный)

⚠️ Важно:

После завершения CABasicAnimation, свойство визуально откатывается — если не зафиксировать финальное значение!

Решение:

```
CATransaction.begin()
CATransaction.setDisableActions(true)
shapeLayer.strokeEnd = 1
CATransaction.commit()
```

Или:

```
animation.fillMode =.forwards
animation.isRemovedOnCompletion = false
```

(но это хрупкий способ — лучше устанавливать значение напрямую)

Что такое CATransaction?

CATransaction — это **механизм группировки изменений слоев**, предоставляемый Core Animation. Он позволяет:

- Объединить **несколько изменений** в одну атомарную транзакцию
- Настроить **параметры анимации по умолчанию** (время, тайминг)
- **Отключить автоматическую анимацию**, если нужно сделать резкое изменение

Зачем мы ее использовали?

```
// Начинается транзакция
CATransaction.begin()
// Отключает автоматическую анимацию
CATransaction.setDisableActions(true)
// toValue - устанавливает нужное значение мгновенно
fgLayer.strokeEnd = toValue
// Завершает транзакцию
CATransaction.commit()
```

Без этой обертки, если просто написать: `fgLayer.strokeEnd = toValue` то Core Animation по умолчанию будет автоматически анимировать такие изменения. Это может конфликтовать с анимацией заданной вручную.

Что произойдет без обертки в CATransaction?

1. CABasicAnimation запустится от fromValue до toValue
2. Затем strokeEnd = toValue начнет свою автоматическую анимацию
3. Получаем двойную анимацию — с дерганием или неправильным эффектом

Когда использовать?

Ситуация	Исп-ть
Установка конечного значения после анимации	✅
Мгновенное изменение CALayer-свойства	✅
Отключение автоматического поведения слоев	✅
Анимация делается только через UIView.animate	❌

`CATransaction` — это **точный переключатель**, позволяющий сказать Core Animation:

“Слушай, вот тут — не анимируй, просто установи значение, а остальное я сам возьму на себя”.

Отрисовка тени

Что такое Offscreen Rendering?

Когда CALayer **не может быть отрисован напрямую** и требует **промежуточного буфера** (offscreen context), который обрабатывается отдельно GPU или даже CPU.

🚨 **Минусы:**

- Медленнее
- Увеличивает расход памяти
- Может вызывать лаги при скролле и анимациях

Пример: правильная тень

```
let path = UIBezierPath(
    roundedRect: bounds,
    cornerRadius: 12
)
layer.shadowPath = path.cgPath
layer.shadowOpacity = 0.3
layer.shadowRadius = 10
layer.shadowOffset = CGSize(width: 0, height: 4)
```

Т-Банк — финансовая онлайн-экосистема, объединяющая полный спектр финансовых услуг для частных лиц и бизнеса